

ATS AI Model — Architecture, Deployment & On-Device Processing Guide

Version: T2 (INJECTION-T2 validated) **Model file:** ats_unified_minilm_int8.tflite
· 22.8 MB **Encoder:** sentence-transformers/all-MiniLM-L6-v2 **Date:** 2026-05-12

Table of Contents

- 1. [What This System Does](#)
- 2. [Model Architecture — The Big Picture](#)
- 3. [How the Model Was Built \(Training Side\)](#)
- 4. [From Keras to TFLite — How It Gets Packaged for Mobile](#)
- 5. [The flutter_deploy/ Package — What's Inside](#)
- 6. [How Flutter Loads and Runs the Model](#)
- 7. [Step-by-Step On-Device Processing Flow](#)
- 8. [Inputs and Outputs — The Exact Contract](#)
- 9. [How to Integrate Into Your Flutter App](#)
- 10. [Validating Your Integration](#)
- 11. [Performance Numbers](#)
- 12. [Common Mistakes to Avoid](#)
- 13. [Quick Reference — Tensor Cheat Sheet](#)

1. What This System Does

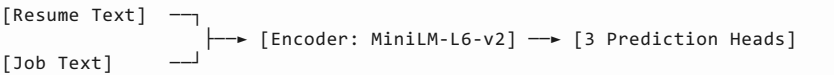
The ATS AI model takes two pieces of text — a **resume** and a **job description** — and tells you three things:

Output	What It Means
ATS Score (0–100%)	How well the resume matches the job description
Domain	Which job category the role belongs to (e.g. IT, Healthcare)
RSG Template (0–45)	Which resume summary template fits best

Everything runs **on the device**. No internet connection needed. No data leaves the phone. The full model is a single `.tflite` file that ships inside your Flutter app.

2. Model Architecture — The Big Picture

Think of the model as a pipeline with three stages:



Stage 1 — Tokenizer (Text → Numbers)

Before the model can read text, the text has to become numbers. This is done by a **tokenizer** — specifically a BERT-style WordPiece tokenizer backed by a vocabulary of **30,522 words and word-pieces**.

- Every word gets split into known pieces (e.g. "developer" → developer, "##er")
- Each piece maps to a number (e.g. developer → 9722)
- The number sequence is always padded or trimmed to exactly **128 positions**

- Special markers are added: [CLS] (ID 101) at the start, [SEP] (ID 102) at the end, [PAD] (ID 0) to fill empty slots

This produces two arrays per text — `input_ids` and `attention_mask` — each of length 128.

Stage 2 — Encoder (Numbers → Meaning)

The tokenized numbers go into the **MiniLM-L6-v2 encoder** (a compact BERT-family transformer). The encoder reads the full sequence of 128 tokens and produces a single dense vector of **384 numbers** that captures the meaning of the text.

This encoder is **frozen** — its weights were pre-trained by Sentence Transformers on a huge corpus and do not change during ATS training.

Stage 3 — Three Prediction Heads

From the two 384-dim vectors (one for resume, one for job description), three separate mini-networks make predictions:

Head 1 — ATS Score - Uses both embeddings - Calculates cosine similarity and dot product between them - Combines all features through two dense layers - Final layer: sigmoid → a number between 0.0 and 1.0 - Multiply by 100 to get the percentage

Head 2 — Domain Classifier - Uses the job description embedding only (the domain is a property of the job, not the candidate) - Two dense layers → softmax over 7 classes - `argmax` of the 7 outputs gives the domain index

Head 3 — RSG Template Selector - Similar structure, trained to pick one of 46 resume summary templates - Two dense layers → softmax over 46 classes - `argmax` gives the template index

3. How the Model Was Built (Training Side)

This section is for understanding history. The Python training code lives in `ats-ai-core/`.

Training Configuration (from `ats-ai-core/src/config.py`)

Parameter	Value
Encoder	all-MiniLM-L6-v2 (384-dim, frozen)
Batch size	32
Epochs	60
Learning rate	0.0001
Sequence length	128 tokens

Loss Weights

The model trains three tasks at the same time. Each task gets a share of attention:

Task	Loss weight
ATS score (regression)	35%
Domain classification	35%
RSG template selection	30%

Training Stages

The model went through several improvement rounds:

- **B-1:** Swapped encoder from USE Lite v2 to MiniLM-L6-v2
- **B-2:** Tuned the ATS + Domain heads
- **B-3:** Added and stabilised the RSG head (recovered to >92% accuracy)
- **B-4:** Converted from Keras to TFLite

Final Metrics Achieved

Metric	Value	Target
ATS MAE	4.15	< 8.0
ATS Band Accuracy	85.9%	—
Domain Macro F1	0.882	> 0.85
RSG Accuracy	92.05%	—
Model size	22.8 MB	< 30 MB

4. From Keras to TFLite — How It Gets Packaged for Mobile

The trained Keras model (`best_unified_weights.h5`) cannot run on a phone directly — it is over 1 GB and needs Python + TensorFlow. To make it phone-friendly, it gets converted to **TFLite format**.

What happens during conversion

1. The full Keras model (encoder + three heads) is exported as a `SavedModel`
2. TFLite converter reads the `SavedModel` and produces a `.tflite` binary
3. **Dynamic-range INT8 quantization** is applied to the weights — this shrinks the file by ~4× with almost no accuracy loss
4. The result: `ats_unified_minilm_int8.tflite` at **22.8 MB**

Key properties of the TFLite file

- Runs on **CPU only** — no GPU delegate, no NNAPI required
- Uses only **standard TFLite built-in ops** — zero custom ops or Flex ops
- Compatible with Android and iOS out of the box
- Weights are INT8; activations remain FLOAT32 (dynamic-range quantization)

5. The `flutter_deploy/` Package — What's Inside

This folder is the complete, ready-to-ship Flutter integration package. Everything you need is here.

```

flutter_deploy/
├── assets/
│   ├── ats_unified_minilm_int8.tflite ← The TFLite model (22.8 MB)
│   └── vocab.txt ← Vocabulary file (30,522
entries)
│   └── tokenizer_config.json ← Tokenizer settings reference
├── lib/
│   ├── ats_tokenizer.dart ← Converts text to token arrays
│   ├── ats_inference_engine.dart ← Loads model, runs inference
│   ├── ats_result.dart ← Result data class
│   └── ats_service.dart ← Orchestration layer
├── validation/
│   └── t2_test_vectors.json ← 20 reference input/output
pairs
│   └── t2_integration_report.json ← Validation report (T2 PASSED)
├── IO_SCHEMA.md ← Full technical IO contract
├── pubspec_snippet.yaml ← pubspec additions needed
└── README.md ← Flutter integration guide

```

What each folder does

assets/ — The three files that ship inside the app bundle. The `.tflite` file is the model brain. `vocab.txt` is what the tokenizer reads to map words to numbers. `tokenizer_config.json` is a reference copy (the app reads `vocab.txt` directly, not this file).

lib/ — Ready-to-use Dart code. Copy these files into your project and they are ready to go.

validation/ — 20 pre-computed test cases with known inputs and expected outputs. Use these to confirm your integration is working correctly.

6. How Flutter Loads and Runs the Model

The Flutter app uses the `tflite_flutter` package to talk to the TFLite runtime. Here is the setup:

Dependencies needed (pubspec.yaml)

```

dependencies:
  tflite_flutter: ^0.10.4
  tflite_flutter_helper: ^0.4.1

flutter:
  assets:
    - assets/ats_unified_minilm_int8.tflite
    - assets/vocab.txt
    - assets/tokenizer_config.json

```

Android build fix

Add this to your `android/app/build.gradle` — without it, Android compresses the `.tflite` file and it fails to load:

```

android {
  aaptOptions {
    noCompress 'tflite'
  }
}

```

Loading the model (Dart)

```
final engine = await AtsInferenceEngine.load();
```

This does two things: 1. Opens `ats_unified_minilm_int8.tflite` from assets and loads it into a TFLite Interpreter with 4 threads 2. Loads the vocabulary from `vocab.txt` into a BertTokenizer

Do this once — at app startup or when the user's screen initialises. Keep the engine alive in memory; do not reload it for every score call.

7. Step-by-Step On-Device Processing Flow

Here is exactly what happens every time you call `engine.score(resumeText, jdText)`:

Step 1 – Tokenize the resume

Input : "Python developer with AWS experience..."

Action: Lowercase → WordPiece split → map to IDs → pad to 128

Output: `resume_input_ids [128 ints]`, `resume_attention_mask [128 ints]`

Step 2 – Tokenize the job description

Input : "We are looking for a Python backend engineer..."

Action: Same process as Step 1

Output: `jd_input_ids [128 ints]`, `jd_attention_mask [128 ints]`

Step 3 – Feed into TFLite interpreter

Pack the 4 arrays into the correct tensor slots:

Slot 0 → `resume_input_ids`

Slot 1 → `jd_input_ids`

Slot 2 → `jd_attention_mask`

Slot 3 → `resume_attention_mask`

Step 4 – Run inference

The interpreter runs the model on the CPU.

Takes ~22 ms on a modern phone.

Step 5 – Read the three outputs

Output 0 [1,1] → raw float (e.g. 0.724) → multiply by 100 → ATS score (72.4%)

Output 1 [1,7] → 7 floats (domain probabilities) → argmax → domain index

Output 2 [1,46] → 46 floats (template probabilities) → argmax → RSG template

Step 6 – Return `AtsResult`

```
AtsResult {  
  atsScore:    72.4,  
  scoreBand:   "Good Match",  
  domainIndex: 0,  
  domainName:  "IT / Software",  
  rsgTemplate: 4  
}
```

All of this happens locally on the device. Nothing is sent over the network.

8. Inputs and Outputs — The Exact Contract

Input tensors (what you feed into the model)

Slot	Tensor name	Shape	Type	What it contains
0	serving_default_resume_input_ids:0	[1, 128]	INT32	Resume token IDs
1	serving_default_jd_input_ids:0	[1, 128]	INT32	Job description token IDs
2	serving_default_jd_attention_mask:0	[1, 128]	INT32	JD attention mask
3	serving_default_resume_attention_mask:0	[1, 128]	INT32	Resume attention mask

Rules: - Batch size is always 1. The model does not support batching. - All arrays are exactly 128 values — no more, no less. - token_type_ids (used in some BERT models) are handled internally — do not add them as inputs. - Attention mask: 1 for real tokens (including [CLS] and [SEP]), 0 for padding.

Output tensors (what you get back)

Slot	Tensor name	Shape	Type	How to use
0	StatefulPartitionedCall:0	[1, 1]	FLOAT32	Multiply by 100 → ATS score (0–100)
1	StatefulPartitionedCall:1	[1, 7]	FLOAT32	argmax → domain index (0–6)
2	StatefulPartitionedCall:2	[1, 46]	FLOAT32	argmax → RSG template index (0–45)

Domain index mapping

Index	Domain name
0	IT / Software
1	Non-IT / Management
2	Design / Creative
3	Healthcare
4	Finance / Banking
5	Legal
6	Education

Score band labels

Score range	Label
85 – 100	Excellent Match
65 – 84	Good Match
45 – 64	Moderate Match
25 – 44	Weak Match
0 – 24	Poor Match

9. How to Integrate Into Your Flutter App

Step 1 — Copy the assets

Copy these three files from flutter_deploy/assets/ into your Flutter project:

```
your_flutter_app/  
  assets/  
    ats_unified_minilm_int8.tflite  
    vocab.txt  
    tokenizer_config.json
```

Step 2 — Update pubspec.yaml

```
flutter:  
  assets:  
    - assets/ats_unified_minilm_int8.tflite  
    - assets/vocab.txt  
    - assets/tokenizer_config.json  
  
dependencies:  
  tflite_flutter: ^0.10.4  
  tflite_flutter_helper: ^0.4.1
```

Step 3 — Copy the Dart files from flutter_deploy/lib/

The four files in `flutter_deploy/lib/` are ready to use:

- [`ats\_tokenizer.dart`](#) — handles text → token arrays
- [`ats\_inference\_engine.dart`](#) — runs the model
- [`ats\_result.dart`](#) — the result data class
- [`ats\_service.dart`](#) — orchestrates all the pieces

Step 4 — Initialise once and score

```
// Initialise once (e.g. in initState or a provider)  
final engine = await AtsInferenceEngine.load();  
  
// Score a resume against a job description  
final result = engine.score(resumeText, jobDescriptionText);  
  
print(result.atsScore);      // e.g. 72.4  
print(result.scoreBand);     // e.g. "Good Match"  
print(result.domainName);    // e.g. "IT / Software"  
print(result.rsgTemplate);    // e.g. 4
```

Step 5 — Android build fix

In `android/app/build.gradle`:

```
android {  
  aaptOptions {  
    noCompress 'tflite'  
  }  
}
```

10. Validating Your Integration

Before shipping, run the 20 test cases from [`flutter\_deploy/validation/t2\_test\_vectors.json`](#).

Each test case has the structure:

```

{
  "id": 1,
  "domain": 0,
  "domain_name": "IT / Software",
  "resume_text": "...",
  "jd_text": "...",
  "tokenization": {
    "resume_input_ids": [ ... 128 integers ... ],
    "resume_attention_mask": [ ... 128 integers ... ],
    "jd_input_ids": [ ... 128 integers ... ],
    "jd_attention_mask": [ ... 128 integers ... ]
  },
  "outputs": {
    "ats_score_pct": 72.4,
    "domain_argmax": 0,
    "rsg_template_argmax": 4
  }
}

```

Acceptance criteria

What to check	Allowed difference
ATS score	Within ± 2 points of reference
Domain index	Exact match
RSG template index	Exact match

Tip: isolate tokenizer vs inference bugs

The test vectors include pre-tokenized tokenization arrays. You can feed those directly into the interpreter (skipping your Dart tokenizer) to check if inference itself is correct. If inference passes but end-to-end fails, the bug is in your tokenizer.

The full validation report is at [flutter_deploy/validation/t2_integration_report.json](#) — all 20 cases passed at stage T2.

11. Performance Numbers

These were measured during T2 validation on CPU:

Metric	Value
Mean inference latency	21.9 ms
95th percentile latency	23.6 ms
99th percentile latency	24.4 ms
Domains tested	7 / 7
Fresher profiles tested	4
Flex ops in model	0
GPU / NNAPI delegate needed	No

The model runs entirely on the CPU. No GPU or NNAPI delegate is needed or recommended — attaching one will not improve speed and may cause errors.

12. Common Mistakes to Avoid

Do not attach a GPU or NNAPI delegate. The model uses only CPU-compatible ops. Forcing a GPU delegate may cause it to fail.

Do not try to run batches. Batch size is hardcoded to 1 inside the model. Always pass one resume and one job description at a time.

Do not skip the noCompress 'tflite' line in Android. Android will try to compress the .tflite file. Compressed TFLite files cannot be memory-mapped and will fail to load.

Do not read tokenizer_config.json for token IDs. The tokenizer reads vocab.txt only. tokenizer_config.json is a reference document.

Do not use SentencePiece or any other tokenizer. The model was trained with **WordPiece** (BERT-style). Using a different tokenizer will produce wrong token IDs and garbage outputs. The tokenizer must use do_lower_case=true.

Do not reload the model on every call. Loading the interpreter takes time. Load it once, keep it in memory, and reuse it for every score call.

Do not forget [CLS] and [SEP] tokens. Every sequence must start with ID 101 ([CLS]) and end with ID 102 ([SEP]). The attention mask must be 1 for these positions.

13. Quick Reference — Tensor Cheat Sheet

INPUTS (feed 4 arrays, all INT32, shape [1, 128])

Slot 0	→ resume input IDs	(token IDs for resume text)
Slot 1	→ job description IDs	(token IDs for JD text)
Slot 2	→ JD attention mask	(1=real token, 0=padding)
Slot 3	→ resume attention mask	(1=real token, 0=padding)

OUTPUTS (read 3 arrays)

Slot 0	→ [1, 1]	FLOAT32	→ value × 100	= ATS score 0-100
Slot 1	→ [1, 7]	FLOAT32	→ argmax	= domain index 0-6
Slot 2	→ [1, 46]	FLOAT32	→ argmax	= RSG template 0-45

TOKENIZER RULES

Vocabulary	: 30,522 WordPiece tokens (vocab.txt)
[CLS] ID	: 101 (start of every sequence)
[SEP] ID	: 102 (end of every sequence)
[PAD] ID	: 0 (fill remaining positions)
[UNK] ID	: 100 (unknown word)
Sequence len	: always 128 (truncate if longer, pad if shorter)
Case	: lowercase all text before tokenizing

FLUTTER PACKAGES

tflite_flutter:	^0.10.4
tflite_flutter_helper:	^0.4.1

Prepared by ATS AI Team · INJECTION-T2 stage · 2026-05-12